Microservices Cheat Sheet

1 Monolith and microservices

Term	Meaning	Learnly example
Monolith	One codebase, one deployable unit, one database. Simple to build, run and test; hurts when many teams and much traffic arrive.	All 12 features in one Spring Boot application
Microservice	A small, independent service built around one business capability. It runs on its own, owns its data and talks to others over the network.	Enrollment service, Payment service
Microservice architecture	An application built as a set of microservices instead of one unit.	Six services instead of one application
Modular monolith	One deployable unit with clear modules and rules about who may call whom. A good step before services.	Catalog, Enrollment, Payment as modules
Deployable	A unit that is built, tested and released as a whole (a jar or war).	One jar for the monolith, one per service
Capability (business capability)	Something the business does for its customers, named with a verb.	Enroll a student, issue a certificate
Team ownership	One team builds, runs and decides how a service changes ('you build it, you run it').	Finance team owns Payment
Distributed monolith	Services that must be released together or share a database: all the cost of services, none of the benefit.	Six services reading one database
Strangler approach	Move from a monolith by extracting one context at a time behind a front door, then repeating.	Extract Notification first, then Payment

Monolith or microservices?

- **Consider microservices when:** several teams must release independently; parts need different scaling; a failure in one part must not stop the rest; the business areas are well understood; you can run and monitor many services
- **Stay with a monolith when:** one small team owns the product; the idea is still changing every week; you cannot yet say where the boundaries are; little experience with distributed systems
- **Rule of thumb:** start as a monolith with clear modules; extract a service when you can name the problem it solves

Benefits	Costs
Change independently; scale what is busy; team autonomy; fault isolation; technology freedom; smaller to understand	Distributed data (no joins, no single transaction); the network (slow, can fail, arrive twice); more to operate and monitor; harder testing; consistency across services; harder to see the whole

2 Design principles

Principle	In one line	Quick test
1 Single responsibility	A service has one reason to change.	Describe it in one sentence without 'and'
2 High cohesion	Things that change together live together.	A business rule change touches one service
3 Loose coupling	A change in one service rarely forces a change in another.	Only a small, stable contract between services
4 Autonomy	A service can be deployed, changed and can fail on its own.	Does A need B to ship or to start?
5 Own your data	One owner per table; database per service; others ask by id.	Can another service read my tables? It must not
6 API is the contract	Public: operations, messages, errors. Private: tables, classes, frameworks.	Could I change the inside without telling anyone?
7 Design for failure	Any call can be slow, fail or arrive twice.	What does the user see if the other side is down?

Principle	In one line	Quick test
8 Team ownership	Services follow teams; one service, one owning team.	Who is called at night when it breaks?
9 Keep it simple	Right size: not too big, not too small. Start bigger, split later.	Can I say why it must be separate?

Three boundary tests (a good boundary passes all three)

Boundary test	Question	Example (Enrollment)
One sentence	Can you describe the service in one sentence without 'and'?	Enrollment decides who may take which course.
One reason to change	Does one team ask for changes, without another service changing?	Finance changes refund rules: only Payment changes
Own data and failure	Does it own its tables and keep working, reduced, when others are down?	Notification is down: enrollment still works

Right size

- **Too big:** many reasons to change, several teams inside, releases wait for each other
- **Right size:** one sentence, one team, owns its data, works (reduced) when others fail
- **Too small:** one feature needs five calls, services cannot work alone, more services than people

3 Domain-Driven Design in plain words

Term	Meaning	Learnly example
DDD (Domain-Driven Design)	Understand the business first, together with the people who know it, then build the software around it. Gives natural service boundaries.	Developers and admissions staff model enrollment together
Domain	The business area the software serves.	Online learning
Domain expert	A person who knows the business well.	Head of admissions

Term	Meaning	Learnly example
Strategic design	The problem space: what is the business and why. Language, subdomains, bounded contexts, context map.	Finding the six contexts
Tactical design	The solution space: how to build each part. Entities, aggregates, events, application services.	Describing what is inside Enrollment
Ubiquitous language	One word, one meaning, used by business people and developers in talk, code, tables and APIs.	'Enrollment' everywhere, never 'registration'
Subdomain	One smaller area of the domain.	Course catalog, payment
Core subdomain	Why customers choose you. Build it well, in-house.	Course catalog, learning progress
Supporting subdomain	Needed but not special. Keep it simple.	Enrollment, payment
Generic subdomain	Every business has it. Buy or reuse a ready-made tool.	Identity, notification, audit log
Bounded context	A boundary inside which every word has one precise meaning. A natural candidate for one microservice.	'Course' means different things in Catalog, Enrollment and Payment
Context map	A drawing of the contexts and how they depend on each other. Each arrow becomes a call or a message.	Six boxes, four asks, a few events
Event storming	A workshop: write business events (past tense) on sticky notes in time order, then look for clusters.	StudentRegistered, PaymentCompleted

Where do we cut? By layer or by business

- **Cut by layer (wrong):** UI service, business service, data service. One business rule touches all three
- **Cut by business (right):** Course Catalog, Enrollment, Learning Progress. The same rule touches one service

4 Inside a service (light tactical design)

Term	Meaning	Learnly example
Entity	An object with an identity that lasts over time.	Course c-305, Student s-77
Value object	An object with no identity, only values; equal values are the same thing.	Money (500, INR), Address
Aggregate	A group of objects that must stay consistent together, changed as one unit.	Payment with Refund and Invoice
Aggregate root	The one entity through which an aggregate is reached from outside.	Payment
Repository	Stores and loads one kind of aggregate (Spring Data repository in Module 22).	EnrollmentRepository
Domain event	Something important that happened, named in the past tense. Becomes a message between services.	PaymentCompleted
Application service	The entry point for a use case: loads the aggregate, asks it to act, saves, publishes events. Rules stay in the domain.	EnrollmentApplicationService
Service worksheet	One page per service: purpose, responsibilities, does not do, entities, aggregate root, asks, events, application services, team.	The Enrollment worksheet

5 Dependencies between services

Term	Meaning	Use when / cost
Ask (call and wait)	One service calls another and waits for the answer.	Use when you need the answer right now. Cost: if the other side is down you wait or fail.

Term	Meaning	Use when / cost
Tell / be told (message)	A service sends a message (event) when something happened; others react.	Use when the other side can act a little later. Cost: the news may arrive late.
Reference by id	Keep only the id of data owned elsewhere, never its table or a full copy.	studentId in Enrollment, not the student record
Loop	A asks B and B asks A (directly or through others).	Never allowed: replace one arrow with a message
Chain of asks	A asks B, which asks C ...	Keep it at most two long

6 The method in six steps

- **Understand** the business: list features and pains with the domain experts
- **Language:** write the glossary (one word, one meaning)
- **Subdomains:** sort into core, supporting, generic
- **Contexts:** group features, name each, one-line responsibility; test with the three boundary tests
- **Map:** draw the context map; mark every arrow ask or be told; no loops
- **Describe and check:** one worksheet per service; review with the checklist